Assignment_7 矩阵、贪心

完成学生：庞又玮 物理学院

1. 题目

M12560：生存游戏

matrices, http://cs101.openjudge.cn/pctbook/M12560/

```
n, m = map(int, input().split())
ini_matrix = [[0] + list(map(int, input().split())) + [0] for _ in range(n)]
matrix = [[0] * (m + 2)] + ini_matrix + [[0] * (m + 2)]
copied_matrix = [row[:] for row in matrix]

DIRS8 = tuple((di, dj) for di in (-1, 0, 1) for dj in (-1, 0, 1) if (di, dj)
!= (0, 0))
for i in range(1, n + 1):
    for j in range(1, m + 1):
        alive = sum(matrix[i + d_i][j + d_j] for d_i, d_j in DIRS8)
        if matrix[i][j] == 1:
            if alive < 2 or alive > 3:
                copied_matrix[i][j] = 0
        else:
            if alive == 3:
                copied_matrix[i][j] = 1

for _ in range(1, n + 1):
    print(*copied_matrix[_][1:m + 1])
```

代码运行截图 （至少包含有"Accepted"）


CS101 / 计算思维算法实践

题目 排名 状态 提问

#53539582提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
n, m = map(int, input().split())
ini_matrix = [[0] + list(map(int, input().split())) + [0] for _ in range
matrix = [[0] * (m + 2)] + ini_matrix + [[0] * (m + 2)]
copied_matrix = [row[:] for row in matrix]

DIRS8 = tuple((di, dj) for di in (-1, 0, 1) for dj in (-1, 0, 1) if (di
for i in range(1, n + 1):
    for j in range(1, m + 1):
        alive = sum(matrix[i + d_i][j + d_j] for d_i, d_j in DIRS8)
        if matrix[i][j] == 1:
            if alive < 2 or alive > 3:
                copied_matrix[i][j] = 0
        else:
            if alive == 3:
                copied_matrix[i][j] = 1

for _ in range(1, n + 1):
    print(*copied_matrix[_][1:m + 1])
```

基本信息

#: 53539582
题目: M12560
提交人: 26n2600011560
内存: 4084kB
时间: 41ms
语言: Python3
提交时间: 2026-10-20 16:32:40

©2002-2022 POJ 京ICP备20010980号-1

English 帮助 关于

M04133：垃圾炸弹

matrices, http://cs101.openjudge.cn/pctbook/M04133/

```
d = int(input())
n = int(input())
trashes = {}
x_max = 0
x_min = 0
y_max = 0
y_min = 0
for _ in range(n):
    x, y, i = map(int, input().split())
    x_max = max(x_max, x)
    x_min = min(x_min, x)
    y_max = max(y_max, y)
    y_min = min(y_min, y)
    trashes[(x, y)] = i
c = x_max - x_min + 1 + 2 * d
r = y_max - y_min + 1 + 2 * d
matrix = [[0] * c for _ in range(r)]

DIRS = tuple((di, dj) for di in range(- d, d + 1) for dj in range(- d, d + 1))
for (x, y) in trashes.keys():
    for d_i, d_j in DIRS:
```

```
            matrix[y - y_min + d + d_i][x - x_min + d + d_j] += trashes[(x, y)]
ans_2 = 0
for i in range(r):
    for j in range(c):
        if 0 <= y_min - d + i <= 1024 and 0 <= x_min - d + j <= 1024:
            ans_2 = max(ans_2, matrix[i][j])
ans_1 = 0
for i in range(r):
    for j in range(c):
        if 0 <= y_min - d + i <= 1024 and 0 <= x_min - d + j <= 1024:
            if matrix[i][j] == ans_2:
                ans_1 += 1

print(ans_1, ans_2)
```

注：我这个代码自认的一个优势是并没有真建一个1025 * 1025的矩阵，而是根据x和y的范围建了一个较小的矩阵，节省了空间和时间

代码运行截图 （至少包含有"Accepted"）



M02746：约瑟夫问题

implementation, queue, http://cs101.openjudge.cn/pctbook/M02746/

```
from collections import deque

while True:
    n, m = map(int, input().split())
    if n == 0 and m == 0:
        break

    lst = [_ for _ in range(1, n + 1)]
    monkeys = deque(lst)
    i = 0
    while i < n - 1:
        monkeys.rotate(n - i - m + 1)
        monkeys.popleft()
        i += 1

    print(monkeys[0])
```

or

```
def josephus_formula(a, b):
    """迭代（编号从0开始）"""
    r = 0
    for k in range(2, a + 1):
        # f(1) -> f(2) -> ... -> f(a)
        r = (r + b) % k
    return r + 1  # 莫忘+1


while True:
    n, m = map(int, input().split())
    if n == 0 and m == 0:
        break

    print(josephus_formula(n, m))
```

or

```
def josephus_formula(a, b):
    """递归（编号从1开始）"""
    if a == 1:
        return 1
    return (josephus_formula(a - 1, b) - 1 + b) % a + 1
```

```
while True:
    n, m = map(int, input().split())
    if n == 0 and m == 0:
        break

    print(josephus_formula(n, m))
```

代码运行截图 （至少包含有"Accepted"）



M26976：摆动序列

greedy, dp, http://cs101.openjudge.cn/pctbook/M26976/

```
def sgn(x: int) -> int:
    """便捷的正负判断函数"""
    if x == 0:
        return 0
    elif x > 0:
        return 1
    else:
        return -1
```

```
n = int(input())
nums = list(map(int, input().split()))
diffs = [sgn(nums[i + 1] - nums[i]) for i in range(n - 1)]

marks = diffs.count(0)
diffs = [x for x in diffs if x != 0]
for i in range(len(diffs) - 1):
    if diffs[i] == diffs[i + 1]:
        marks += 1

print(n - marks)
```

or

```
def sgn(x: int) -> int:
    if x == 0:
        return 0
    elif x > 0:
        return 1
    else:
        return -1


n = int(input())
nums = list(map(int, input().split()))
delta = [sgn(nums[i + 1] - nums[i]) for i in range(n - 1)]

result = 1
pre = 0
for i in range(n - 1):
    if delta[i] * pre < 0 or (pre == 0 and delta[i] != 0):
        result += 1
        pre = delta[i]
print(result)
```

or

```
def sgn(x: int) -> int:
    if x == 0:
        return 0
    elif x > 0:
        return 1
    else:
        return -1
```

```
n = int(input())
nums = list(map(int, input().split()))
delta = [sgn(nums[i + 1] - nums[i]) for i in range(n - 1)]

result = 1
pre = 0
for i in range(n - 1):
    if delta[i] * pre < 0 or (pre == 0 and delta[i] != 0):
        result += 1
        pre = delta[i]

print(result)
```

or

```
# dp（动态规划）

n = int(input())
li = list(map(int, input().split()))

dpup = 1  # 当前处理到的位置，最后一段趋势为上升的最长摆动子序列长度
dpdo = 1  # 当前处理到的位置，最后一段趋势为下降的最长摆动子序列长度

for i in range(1, n):
    if li[i] > li[i - 1]:
        dpup = max(dpdo + 1, dpup)
        # dpup = dpdo + 1即可
    if li[i] < li[i - 1]:
        dpdo = max(dpup + 1, dpdo)
        # dpdo = dpup + 1即可

print(max(dpup, dpdo))
```

or

```
def wiggleMaxLength(nums: list) -> int:
    n = len(nums)
    if n == 1:
        return 1
    direction = None
    res = 0
    for i in range(1, n):
        if nums[i] == nums[i - 1]:
```

```
            continue
        elif nums[i] - nums[i - 1] > 0:
            if direction == 1:
                continue
            direction = 1
            res += 1
        else:
            if direction == 0:
                continue
            direction = 0
            res += 1
    return res + 1


input()
*nums, = map(int, input().split())
ans = wiggleMaxLength(nums)
print(ans)
```

代码运行截图 （至少包含有"Accepted"）



T26971：分发糖果

greedy, http://cs101.openjudge.cn/pctbook/T26971/

```python
def sgn(x: int) -> int:
    if x == 0:
        return 0
    elif x > 0:
        return 1
    else:
        return -1


n = int(input())
ratings = list(map(int, input().split()))
delta = [0] + [sgn(ratings[i + 1] - ratings[i]) for i in range(n - 1)] + [0]
candies = [0 for _ in range(n)]
mins = []

for i in range(n):
    if delta[i] <= 0 <= delta[i + 1]:
        mins.append(i)
for i in range(n):
    if i in mins:
        candies[i] = 1
    if delta[i] > 0 and delta[i + 1] >= 0:
        a = max(a for a in mins if a < i)
        candies[i] = i - a + 1
    if delta[i] <= 0 and delta[i + 1] < 0:
        b = min(b for b in mins if b > i)
        candies[i] = b - i + 1
    if delta[i] > 0 > delta[i + 1]:
        a = max(a for a in mins if a < i)
        b = min(b for b in mins if b > i)
        candies[i] = max(i - a + 1, b - i + 1)

print(sum(candies))
```

标准做法（贪心+dp）
深受T4启发呀...

```python
n = int(input())
r = list(map(int, input().split()))

res = [1] * n

for i in range(1, n):
    if r[i] > r[i - 1]:
        res[i] = res[i - 1] + 1
```

```
for j in range(n - 1, 0, -1):  # 倒序遍历
    if r[j - 1] > r[j]:
        res[j - 1] = max(res[j - 1], res[j] + 1)

print(sum(res))
```

代码运行截图 （至少包含有"Accepted"）



1868A : Fill in the Matrix

constructive algorithms, implementation, 1300,
https://codeforces.com/problemset/problem/1868/A

```
t = int(input())

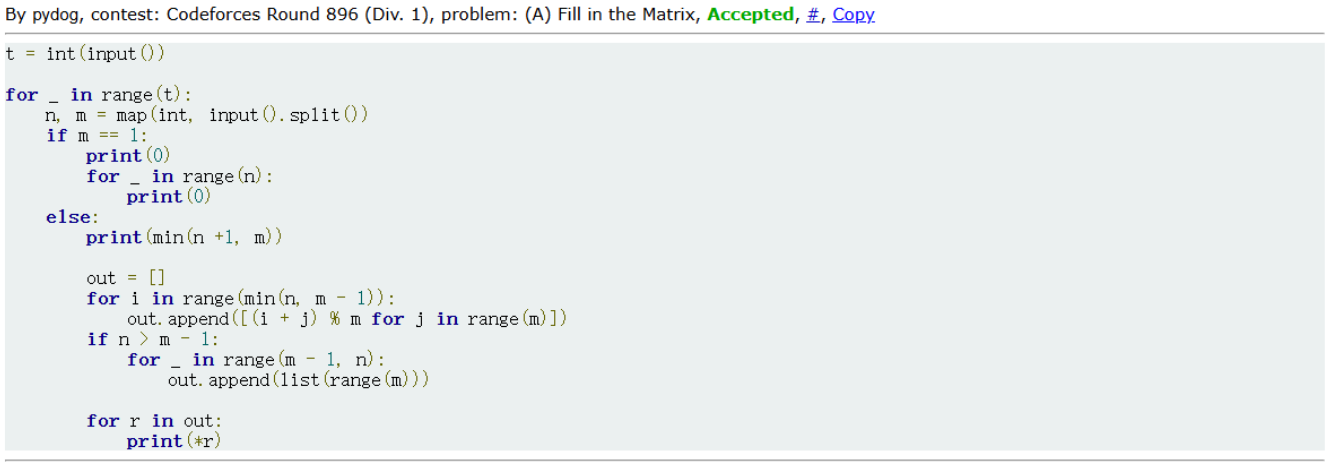
for _ in range(t):
    n, m = map(int, input().split())
    if m == 1:
        print(0)
        for _ in range(n):
            print(0)
```

```
    else:
        print(min(n + 1, m))

        out = []
        for i in range(min(n, m - 1)):
            # 完成“循坏移位”
            out.append([(i + j) % m for j in range(m)])
        if n > m - 1:
            for _ in range(m - 1, n):
                out.append(list(range(m)))

        for r in out:
            print(*r)
```

代码运行截图 （至少包含有"Accepted"）



By pydog, contest: Codeforces Round 896 (Div. 1), problem: (A) Fill in the Matrix, **Accepted**, #, Copy

```
t = int(input())

for _ in range(t):
    n, m = map(int, input().split())
    if m == 1:
        print(0)
        for _ in range(n):
            print(0)
    else:
        print(min(n +1, m))

        out = []
        for i in range(min(n, m - 1)):
            out.append([(i + j) % m for j in range(m)])
        if n > m - 1:
            for _ in range(m - 1, n):
                out.append(list(range(m)))

        for r in out:
            print(*r)
```

2. 学习总结和收获

具体地说：

- 也就T1还算是轻松AC了吧~~~
- T2自己AC但是耗时>1.5h，最大教训是for 键 in 某字典在OpenJudge上会一直Compile Error！必须得写for 键 in 某字典.keys()！
- T3做得很爽：法一用collections.deque；法二迭代；法三递归~~~
- T4本来不是特别不难，但由于亿些原因一直错，饱餐休息后AC啦...
- T5自己通过枚举9种情况的笨办法AC了，看完题解网站上的优秀代码并震惊瘫坐！
- T6构造的关键步骤自己隐隐约约感受到了，半独立地AC了...

笼统地讲，课程难度还是非常大的（从跟室友/高中同学听我说我们在讲什么的时候的震惊神态可知），自己写代码写得还是很煎熬的，很多时候抓耳挠腮弄不明白。另外，觉得这周作业又比上周的难了一个档，比上周做起来难受多了。不过总体上是能感受到进步就好，看到自己写的代码AC了的时候也确实非常开心，同时也在题解网站上看别人的代码收获很大。

最后，求善待！